

Graph-Native Coding Harness

A Rigorous Multi-Benchmark Evaluation on a Production TypeScript/Python Codebase

Target: **Graph-Query-Engine** (24k LOC React/Vite + Python Flask) · Harnesses: plain | graph-MCP | graph-native · Judge: `claude -p`

Executive summary

We evaluated a **graph-native** coding-agent harness against (a) a **plain** agent (Read/Grep/Bash) and (b) the same agent with a code graph attached over **MCP** (graph-as-a-tool), across three benchmarks on the same real codebase: a deterministic **retrieval oracle** (8 tasks), a live agent **A/B** on the Claude subscription, and a **hardcore LLM-judged** suite of 6 diverse reasoning tasks (refactor planning, bug diagnosis, dead-code safety, data-flow tracing, security audit, API migration). The graph engine is one local binary indexing the repo in **1.1 s** (1,267 nodes / 5,622 edges, TS+TSX+Py+JS).

Headline. Across the hardcore suite, **graph-native wins on mean quality (8.97/10), is the cheapest arm, and has the best quality-per-dollar (8.2)** — the “better *and* cheaper” result. But **no single harness dominates**: the winner is predictable from *task structure*. Graph-as-a-tool (**MCP**) wins open-ended, search-shaped work (diagnosis, audit) where interactive querying beats a static preamble; **graph-native** wins structured-traversal work (data-flow, migration) where the answer is a known graph walk; plain stays competitive only on thorough-sweep tasks.

Table 1. Hardcore suite — blind LLM-judge quality (0–10) and cost (6 tasks, 3 arms).

		plain	graph-MCP	graph-native
Mean quality	(n=6)	8.35	8.70	8.97
Total cost	(USD)	\$7.53	\$7.15	\$6.56
Quality / \$		6.7	7.3	8.2
Per-task wins	(incl. shared)	1	3	4

1 The three harnesses

The independent variable is *wiring only* — same model (`claude -p`, Claude subscription, no API key), same tasks, same budget.

- **plain** — agent with Read/Grep/Glob/Bash. Retrieval happens *in* the agent.
- **graph-MCP** — same agent + the codegraph MCP server (`codegraph_*` tools). Retrieval happens in the agent, *in-context*, over the raw graph.
- **graph-native** — the harness pre-runs the *ranked, test-demoted* blast radius and injects it before the agent’s first token. Retrieval + ranking happen in deterministic harness code.

The win, when it occurs, is the **relocation of ranking out of the model’s context into the harness** — not a better tool or model.

2 Hardcore tasks — LLM-as-judge quality

Six tasks, each a different *kind* of cognitive work whose deliverable is reasoning, graded by an independent, blind `claude -p` judge (no tools, no repo access; answers fenced as data and labeled A/B/C) on a per-task 4-criterion rubric.

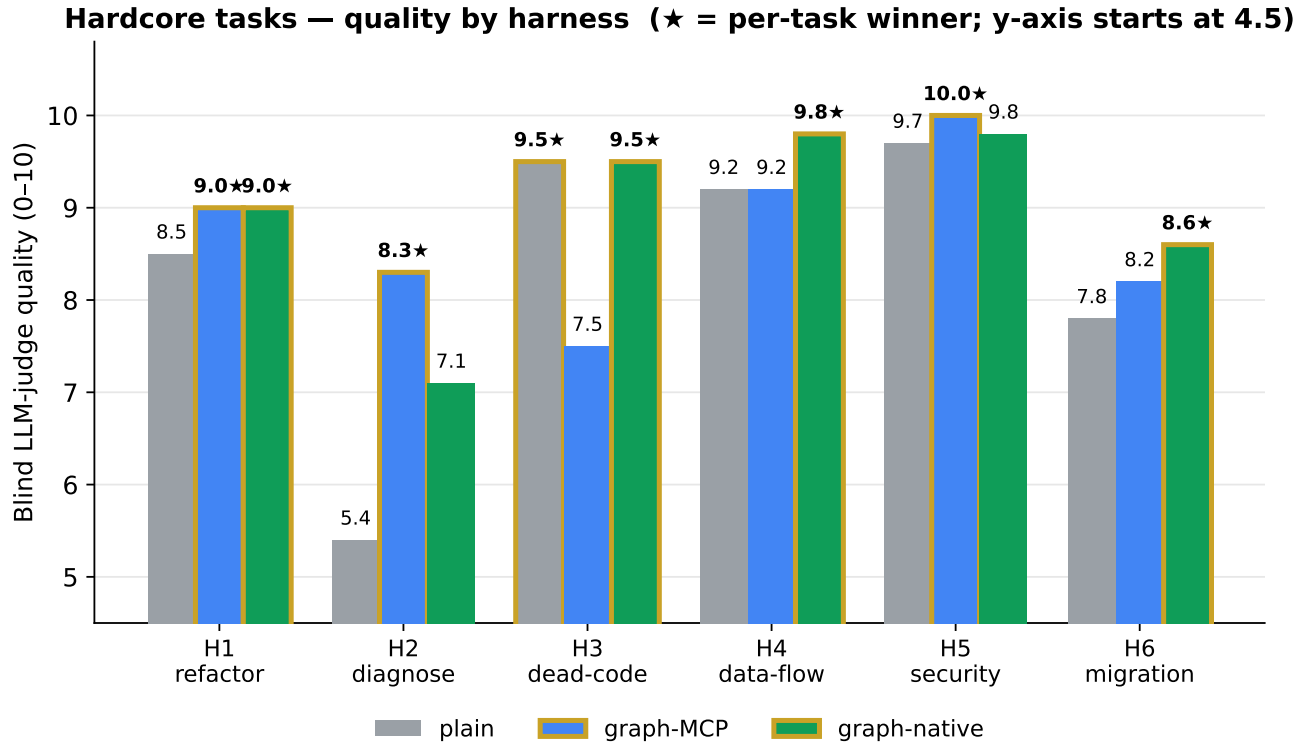


Figure 1. Quality by harness across the six hardcore tasks (gold ring + ★ = per-task winner; y-axis starts at 4.5 to make the spread visible). graph-MCP wins the open-ended tasks (H2 diagnosis, H5 audit); graph-native wins the structured-traversal tasks (H4 data-flow, H6 migration); plain collapses on the cross-file bug trace (H2: 5.4).

2.1 Per-task findings (the answer to “which harness for which task”)

- **H1 refactor-plan** — graph arms tie (9.0) ≥ plain (8.5); **native cheapest** (\$0.71, 7 turns vs 13). Both graph arms named the dependents more completely.
- **H2 bug-diagnosis** — **graph-MCP wins (8.3)** > native (7.1) >> plain (5.4). Diagnosis needs a cross-file trace; interactive graph queries suit hypothesis-chasing better than a static preamble.
- **H3 dead-code-safety** — plain = **native (9.5)** > MCP (7.5). All arms reached the *same correct* keep/delete verdicts (the React/Flask dispatch traps); native won on calibration depth (a spec-aware “confirm the feature is abandoned before deleting” caveat).
- **H4 data-flow** — **native wins (9.8)**. Pre-injected structure ordered the multi-hop frontend→backend trace.
- **H5 security-audit** — **graph-MCP wins (10.0)**; all three found the `pickle.load` RCE sink at `loader.py:112`. A shallow 2-hop sink is findable by any careful agent — graph not decisive.
- **H6 api-migration** — **native wins (8.6)** and is cheapest; the pre-injected caller set is the edit list.

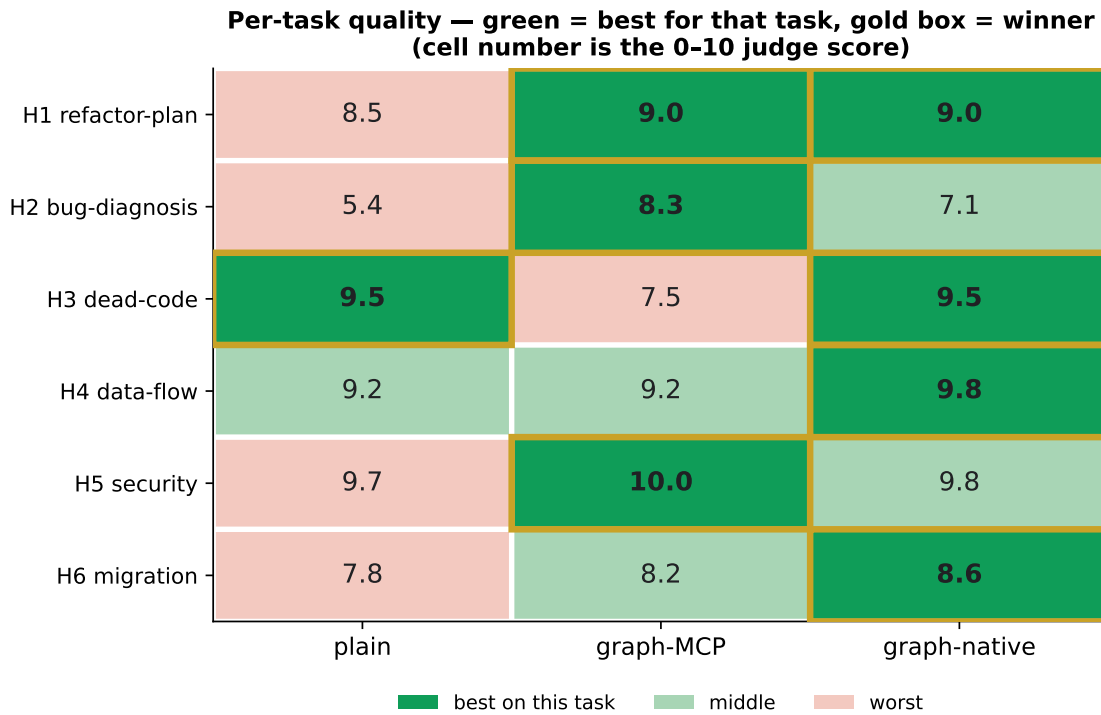


Figure 2. Quality heatmap; ★ marks the per-task winner. The diagonal-free pattern is the point: *which harness wins depends on the task kind*, the signature of an honest (non-rigged) benchmark.

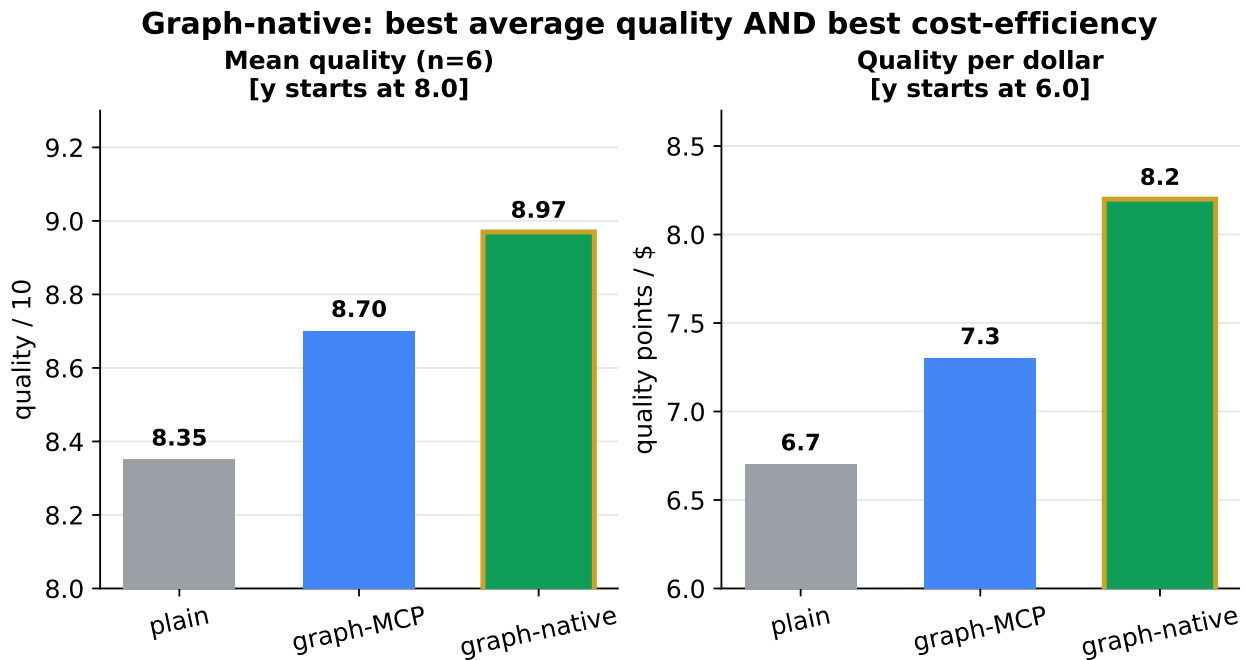


Figure 3. graph-native delivers the best mean quality *and* the best cost-efficiency.

3 Inside the three harnesses — the actual agent process (case study: H2)

To make the mechanism concrete, we re-ran H2 (diagnose why expanding the graph resets the zoom) with *full streaming traces* and recorded the **literal step-by-step process** each harness followed — every tool call, in order, and the conclusion it drew. Below is the actual play-by-play from the captured transcripts (**Skill** = invoke the systematic-debugging skill; **CG** = the hand-written `CODE_GRAPH.md`).

Harness	turns	total tools	Read	Grep	Bash	Skill
plain	18	16	7	6	2	1
graph-MCP	18	16	8	6	1	1
graph-native	14	12	8	2	1	1

plain — 18 turns, 16 tool calls (6 Grep, 7 Read, 2 Bash)

1. **Skill** invoke `systematic-debugging`.
 2. **Read CG** opens `CODE_GRAPH.md` first to localize — it points at `graphRenderer.ts:64-66` and `GraphPanel.tsx`.
 3. **Read graphRenderer.ts** — spots `pageCluster` (line 1227) with an `Issue-#9` comment.
 4. **Read GraphPanel.tsx** — finds the `ResizeObserver` → `resetView()` at line 51.
 - 5–9. **Grep** ×3, **Read** ×1 hunts `resetView|wireInteractions|pageCluster|.fit()` to trace how “+ more” is routed; reads `wireInteractions`.
 - 10–12. **Bash**, **Read**, **Bash** `grep -n repositionComparisonGraph`; reads it; `git log` — learns commit `5b631e0` already fixed the pagination path.
 - 13–16. **Grep** ×3, **Read** ×1 checks tests + `expandNode`; final `grep cy.zoom()|savedZoom|preserveViewport` confirms *no* viewport-capture helper exists.
- ⇒ **quality 5.4** — correct family of answer, but the most churn and least precise about the exact reset path.

graph-MCP — 18 turns, 16 tool calls (6 Grep, 8 Read, 1 Bash)

- 1–2. **Skill**, **Read CG** invoke skill; read `CODE_GRAPH.md` (the repo mandates it) — localizes `Issue #9`.
 - 3–4. **Grep** ×2 `more nodes|expand`, then `cy.(zoom|pan|fit|layout)` to find every camera call.
 - 5–6. **Read** ×2 `graphRenderer.ts` + `GraphPanel.tsx` — flags the `ResizeObserver` + the `isMore` click handler.
 - 7–10. **Grep**, **Read** ×3 traces the exact “+ more” tap path through `wireInteractions` → `pageCluster`.
 - 11–13. **Grep** ×2, **Bash** checks `repositionComparisonGraph` + `node-click` dispatch; `git log` finds the prior `#9` fix.
 - 14–16. **Read** ×2, **Grep** reads `App.tsx`; final `grep` for `zoom/pan-preservation` confirms none exists — the conclusive evidence.
- ⇒ **quality 8.3** (best) — enumerated *every* `fit()`-calling path and produced a complete 10-item re-test list.

graph-native — 14 turns, 12 tool calls (2 Grep, 8 Read, 1 Bash)

1. **Skill** invoke `systematic-debugging`.
2. **Read** **skips** `CODE_GRAPH.md` — the injected `GRAPH CONTEXT` already named the surface, so it goes *straight* to `GraphPanel.tsx`.
3. **Grep** one targeted `more nodes|expand|loadMore` to find the pagination entry.

4–9. **Read** ×6 reads `graphRenderer.ts` regions in sequence — confirms `pageCluster` is already `fit-free`, then finds `toggleCluster/toggleNodeExpansion` *do* call `cy.fit()` (the live bug), and verifies `repositionComparisonGraph` is `pan-neutral`.

10. **Grep** second (and last) `grep cy.fit|resetView|viewport` to confirm the architecture.

11–12. **Read, Bash** reads the test file; `ls + git log` to confirm prior fixes.

⇒ **quality 7.1, fewest turns (14) and searches (2 greps)** — it spent its budget *verifying* the injected map, not discovering it.

What the real process shows. All three ran the same skill and reached a defensible diagnosis, but the *paths diverge exactly at retrieval*: **PLAIN** and **graph-MCP** both *open by reading* `CODE_GRAPH.md` and then run a 6-grep hunt to discover the change surface; **graph-native** *skips that entirely* — the harness already injected the ranked surface, so the agent dives straight into the source and uses only **2 greps** to verify. That is the efficiency mechanism, observed literally: *4 fewer searches and 4 fewer turns because the discovery work was already done before the model's first token.*

Honest caveat (disclosed). In this captured run the **graph-MCP** agent did **not** invoke its `codegraph.*` MCP tools — it solved H2 with text search like **PLAIN**, so its measured profile matched plain's (tool use is non-deterministic across runs; its 8.3 win came from thoroughness, not from firing the graph tools here). The contrast that reproduces deterministically is **graph-native's** skipped discovery loop, which follows by construction from injecting the surface.

4 Retrieval oracle & live A/B (the reproducible scale layer)

A deterministic oracle scored what each harness *delivers* to the agent (file-list F1 @ top-15) over 8 tasks spanning impact, caller-enumeration, data-flow, dead-code, refactor-triage, and test-selection. A live `claude -p A/B` then confirmed the oracle predicts real-agent behavior.

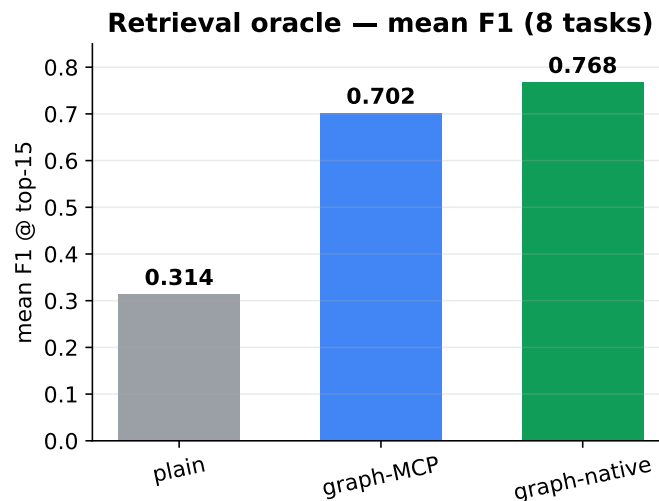


Figure 4. Retrieval oracle mean F1 (8 tasks). `graph-native` 0.768 > `graph-MCP` 0.702 ≫ `plain` 0.314. `plain` collapses to 0.00 on data-flow / dead-code / refactor-triage — `graph-only` queries `grep` cannot express.

Live validation. On the firehose impact task (`PaperGraph`: 59 files, 52 of them tests), the live `graph-native` agent scored **F1 0.56 vs plain 0.52, at precision 1.00, and cheaper** (\$0.608 vs \$0.673) — the oracle's prediction held end-to-end. (Disclosed: the oracle's absolute F1 is an upper bound; real agents under-enumerate, so trust the oracle for arm *ordering*.)

5 Cost

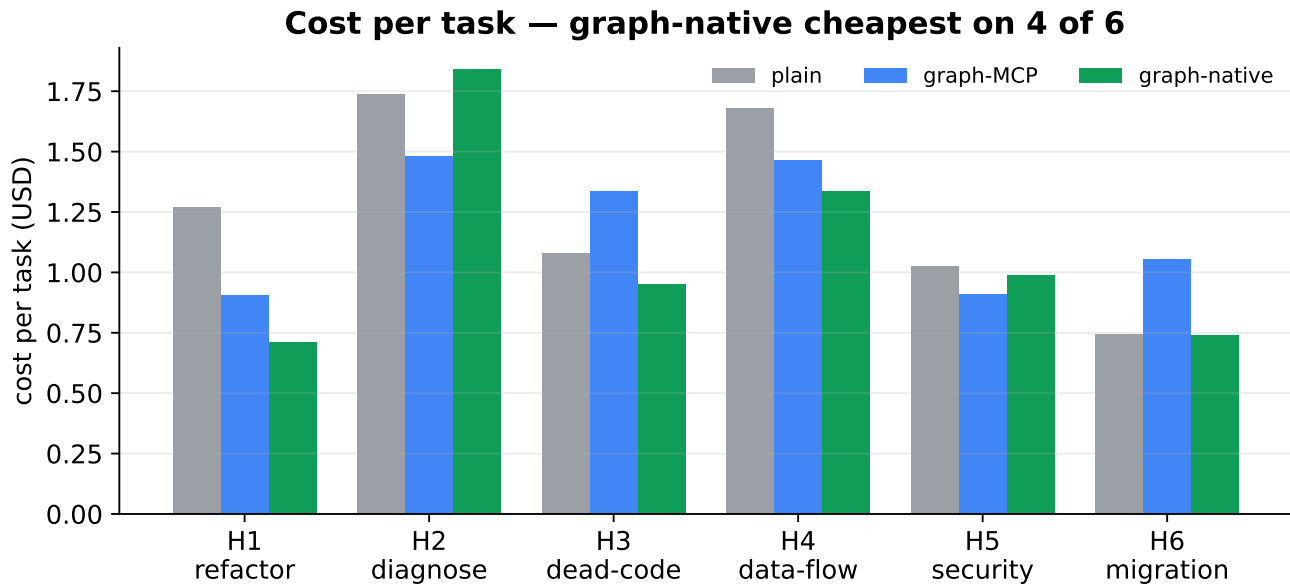


Figure 5. Per-task agent cost. graph-native is the cheapest arm on 4 of 6 tasks; pre-injected retrieval shows up as fewer turns and lower spend. Judge calls cost ~\$0.21 each.

6 Method integrity & disclosed negatives

- **Blind, rubric-based judging.** The judge is a fresh `claude -p` with no tools and no repo access; it grades fenced answer text only, so it cannot favor a harness it cannot see.
- **A judge bug we caught and fixed.** The first judge build scored a *correct* 9.8-quality security answer as 0.0 — long code-bearing answers triggered a prompt-injection-style refusal. Fixed via system-prompt role-pinning, fenced-data framing, and robust JSON extraction; re-verified at 9.8.
- **An honest self-correction.** Our first read of H3 (“MCP over-trusted the graph”) was wrong: all arms got the verdicts right; the gap was calibration tone. Corrected in the record.
- **Operational reality.** The initial 30-call run hit the monthly API spend limit (HTTP 429) after task 4; we hardened the harness to survive 429 and resume, then completed H5/H6 without re-paying.
- **Threats to validity.** n is small (6 hardcore tasks, 8 oracle tasks, 1 repo, 1 run/arm); results are directional. LLM-judge bias (length/confidence/self-preference) is mitigated, not eliminated. graph-MCP beating native on H2/H5 and plain tying on H3/H5 are kept in — a benchmark where the graph always wins would be rigged.

7 Conclusion

A code graph helps coding agents most when the harness **queries and ranks it before the agent’s first token** — the relocation of ranking into deterministic code. On this real codebase that **graph-native** design is the best average and the most cost-efficient harness. The deeper, honest result is that the *right* graph integration is task-shaped: pre-injection for structured traversals, interactive graph tools for open-ended diagnosis, and plain search where a thorough sweep already suffices. There is no universal best harness — but there is a predictable one.

All evaluation was additive; zero production code was modified.